Control Structures - Repetition Statements

Translation Between Repetition Structures

Department of Computer Science
University of Pretoria

2 May 2024

For Loop Overview

A for loop is used when you want to do something for a **known** number of times. The general syntax is as follows:

```
for (initialisation; condition; update) {
    // Code to execute during each iteration
}
```

You should use for loops when you want to:

- Read a known number of values
- Add a known number of values
- Find the maximum/minimum in a list of known size
- Any other suggestions?

While Loop Overview

A while loop is controlled by a condition. The statements in the body of the loop is executed `while` the condition remains true. The general syntax is as follows:

```
while (condition) {
    // statement or block of statements
}
```

You should use while loops when you want to:

- Read an unknown number of values
- Do something until a specific condition occurs
- Search for a value
- Any other suggestions?

Do While Loop Overview

A do-while loop is controlled by a condition. It will always execute the statements **at least once**, since the condition is checked at the end. The statements will then be executed again `while` the condition remains true. The general syntax is as follows:

```
do {
    // statement or block of statements
} while (condition);
```

You should use do-while loops when you want to:

- Display a **menu** and get input from user
- Any other suggestions?

For to While

A for loop has an `initialisation`, `condition` and `update`.

1. The `initialisation` takes place once, at the start of the loop.
2. The `condition` is evaluated each time before the loop is executed
3. The `update` happens at the end of each loop

Using these semantics we can develop an algorithm to translate `for loops` to `while loops`

1. First declare and initialise the loop control variable outside the `while` loop (since this should happen only once)
2. The `condition` from the for loop should be the condition for the `while` loop
3. Add the `for` loop body to the body of the `while` loop
4. At the end of the body of the while loop, add the `update`

A for loop:

```
for (int i = 10; i > 0; i--) {
    cout << i << endl;
}
```

The equivalent while loop:

```
int i = 10; //initialisation before the loop begins
while(i > 0) { // for condition = while condition
    cout << i << endl; //for loop body
    i--;    // for loop update at end of body
}
```

For to Do-While

Similarly for loops can be translated to `do-while` loops.

1. First declare and initialise the loop control variable outside the `do-while` loop (since this should happen only once)
2. Add the `for` loop body to the body of the `do-while` loop
3. At the end of the body of the `do-while` loop, add the `update`
4. The `condition` from the for loop should be the condition for the `do while` loop

A for loop:

```
for (int i = 10; i > 0; i--) {
    cout << i << endl;
}
```

The *almost* equivalent do-while loop:

```
int i = 10;
do {
    cout << i << endl;
    i--;
} while (i > 0);
```

In what cases will this not be equivalent? *Hint: The initial or final value*

may be obtained through user input

While to For

It is **only possible** to write 'while' and 'do-while' loops in terms of a 'for' loops if you know in advance how many times these loops must be executed.

Not all while loops can be translated to for loops, without "abusing" the for loop

1. The while loop should have 1 or more loop control variables that are updated at the end of the loop
2. The while loop conditions should depend on the loop control variables
3. The while loop executes a known/predictable number of times

While loop:

```
int i = 5; // Loop control variable
while (i > 0) { // Condition depends on loop control
    cout << "Number:-" << i << endl;
    i--; //Update of loop control variable
}
```

Equivalent For Loop:

```
for (int i = 5; i > 0; i--) {
    cout << "Number:-" << i << endl;
}
```

Bad Example of While to For

While loop:

```
int userVal = 0;
while (userVal != -1) {
    cout << "Enter a number, -1 to stop:";
    cin>>userVal;
    cout<<"You entered "<<userVal<<endl;
}
```

Equivalent For Loop:

```
for (int userVal = 0; userVal != -1; ) { //Empty update
    cout << "Enter a number, -1 to stop:";
    cin>>userVal;
    cout<<"You entered "<<userVal<<endl;
}
```

Can you predict how many times this while loop will execute? Should you translate it to a for loop?

We are performing a binary search using a `For` loop

```
int target;
cout << "Enter a number between 0 and 100 to search:";
cin >> target;
int low = 0;
int high = 100;

for (; low <= high;) {
    int mid = low + (high - low) / 2;
    if (mid == target) {
        cout << "Number " << target << " found." << endl;
        break;
    } else if (mid < target) {
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

cout << "End of loop" << std::endl;
```

What "hints" are there in the code that this is a bad use case for a for loop? Which repetition structure would be better suited? Translate the for loop into that repetition structure